

プログラム説明資料

KuboEngine / Octopus

C++20 / DirectX 12で制作した自作ゲームエンジンと、
その上で動作する3Dサバイバルシューティング。

OBB + 空間分割

衝突形状と候補削減

Shadow Map

深度描画とGPU計測

FrameContext / SRV

GPU完了を基準にした寿命管理

CSV / CPU Test

調整値と自動確認

TAKU OKUBO / オオクボ タク

ゲームプログラマー志望

目次

設計判断が必要だった箇所を中心に、実装と確認結果を順に示します。

01 作品概要

ゲーム内容 / 担当範囲 / 技術構成

02 全体設計

engineとgameの境界 / scene構成

03 OBB衝突判定

採用理由 / SAT / デバッグ描画

04 候補削減

空間セル / 近傍検索 / CPUテスト

05 Shadow Map

深度描画 / カリング / 調整値

06 リソース寿命

FrameContext / SrvManager / stress

07 CSV・検証

選定理由 / strict parse / telemetry

08 改善履歴と今後

確認済みの範囲 / 次の計測

作品概要

DirectX 12を基礎から扱い、ゲームを一本動かせるところまでを自作しました。



実際のゲーム画面。敵、HUD、ミニマップ、Shadow Mapを同時に確認できます。

ゲーム内容

敵を倒して経験値を集め、通常弾、周囲弾、雷撃、爆発弾や能力値を強化する3Dサバイバルシューティングです。プレイ中のコインはResultシーンで保存データへ反映し、Titleシーンの恒久強化とキャラクター選択に使います。

担当範囲

個人制作。DirectX 12の初期化、描画、入力、音声、モデル、パーティクル、シーン管理から、ゲーム側のプレイヤー、敵、武器、HUD、調整用CSV、デバッグ表示まで実装しました。

エンジン層とゲーム層

engine/

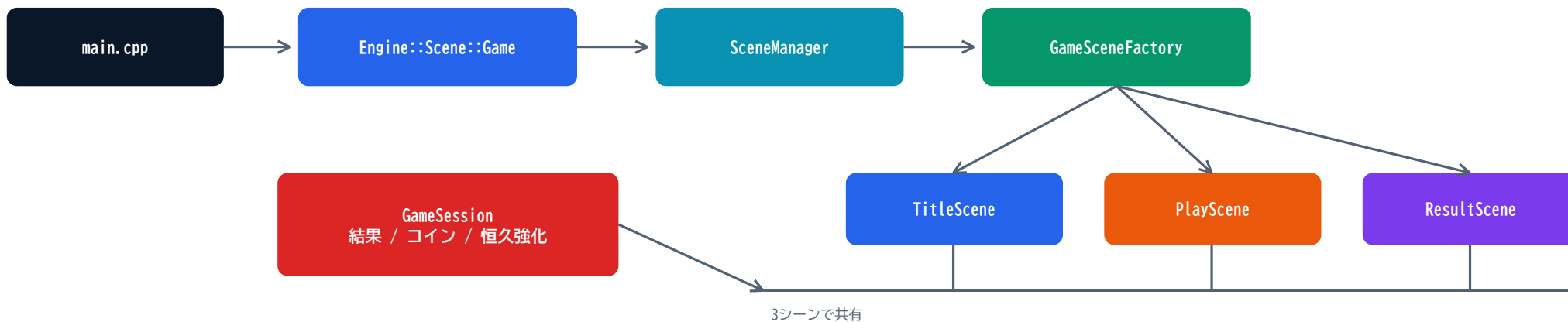
DirectX 12 / FrameContext / Shadow Map / Math / Input / Audio / Model / Particle / Scene

game/directxgame/

Title / Play / Result / Player / Enemy / Weapon / HUD / Effects / Debug / CSV

全体設計

起動処理、シーン生成、ゲーム進行を分け、変更理由が異なる処理を同じクラスへ集めない構成です。



PlaySceneは接続点

PlayScene

プレイヤー、敵、ワールド、HUD、レベルアップUI、ポーズUI、演出、DebugContextを生成して接続します。各機能の中身は専用クラスへ分けています。

GameplayFlowController

Start / Playing / BossIntro / Boss / BossDefeated / Paused / LevelUp / Dead

EnemyManager

敵の生成、EXP、ボス、衝突処理を EnemyCollisionSystemへ接続。

PlayerManager

成長値と武器処理の窓口。 PlayerWeaponControllerへ委譲。

main.cpp - scene生成をfactoryへ渡す

```

auto game = std::make_unique<Engine::Scene::Game>(
    std::make_unique<DirectXGame::GameSceneFactory>(),
    DirectXGame::SceneId::kTitle);
  
```

OBB衝突判定

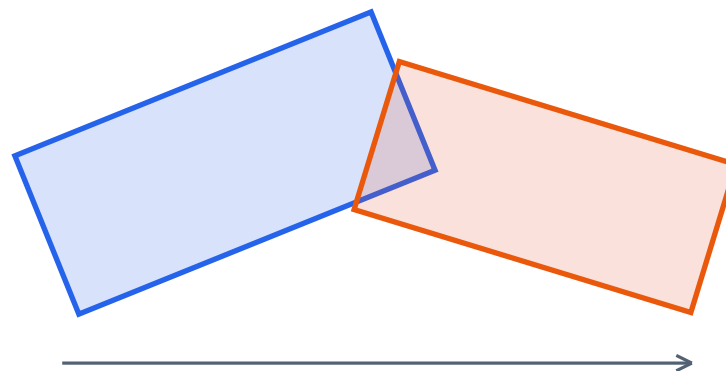
回転する敵や弾で見た目とのずれを減らすため、空間分割後の詳細判定にOBBを使います。

採用理由

球判定は細長い形状で余白が大きくなり、AABBは回転すると見た目より判定範囲が広がります。OBBは向きに沿った判定を作れますが、計算量が増えるため詳細判定に限定しました。

方法	長所	今回の判断
球	距離比較だけ	細長い形状では余白が大きい
AABB	単純で軽い	回転時に判定範囲が広がる
OBB	向きに沿う	空間分割後の詳細判定に採用

分離軸判定（SAT）



15本の候補軸へ投影し、1本でも分離すれば非衝突

MyMath.h / MyMath.cpp

```
struct OBB {  
    Vector3 center;  
    Vector3 orientations[3];  
    Vector3 size;  
};  
constexpr float kEpsilon = 0.0001f;
```

コードで解決している問題

2つのOBBが持つ3軸ずつと、軸同士の外積9軸を調べます。平行に近い軸では計算が不安定になるため、内積の絶対値へkEpsilonを加えています。判定形状はデバッグ描画で確認できます。

OBB判定の候補削減

高コストなOBB判定を全ての敵へ行わず、XZ平面の空間セルから近傍候補だけを集めます。



EnemyCollisionSystem.cpp

```

CollectNearbyEnemies(spatialMap, bulletPosition,
    bulletRadius + kEnemyQueryPadding, nearbyEnemies);
for (Enemy* enemy : nearbyEnemies) {
    if (!MyMath::IsCollision(
        bulletOBB, enemy->GetCollisionOBB())) {
        continue;
    }
    // hit processing
}
  
```

実装上の注意

- 敵の中心だけでなく、衝突半径が重なる全セルへ登録する。
- 複数セルから同じ敵が入るため、並べ替えて重複を除く。
- 負座標を含むセルキーはuint32_t経由で64bit値へ変換する。
- 通常弾、周囲弾、爆発弾、ドローン弾、敵同士の押し戻しで共通利用する。

敵 / query	全走査 ms	空間分割 ms	結果	候補比較回数
128 / 224	0.0356	0.0417	全走査が速い	28,672 → 144
256 / 352	0.1137	0.0847	25.5%短縮	90,112 → 428
512 / 608	0.4212	0.2106	50.0%短縮	311,296 → 1,436
1024 / 1120	1.3107	0.3777	71.2%短縮	1,146,880 → 5,108

Release x64 / Core i7-11800H / cell 8.0 / fixed seed / 120 samples

512体では重複許容0.1696ms、sort/uniqueあり0.2106ms。ただし許容時はヒットが192件から280件へ二重計上されたため、正しさを優先して除外しています。値は広域判定を切り出した測定です。

Shadow Map

モデル形状とライト方向を反映した影を描くため、通常描画とは別にライト視点の深度を作ります。



実画面。敵とプレイヤーの影が地面に落ちる。



2048x2048の深度テクスチャを作成し、R32_FLOATのSRVとして通常描画へ渡します。Object3D::DrawShadowはcastsShadowとライト視錐台を確認し、範囲内だけを送信します。

方法	負荷	表現	判断
影なし	最小	接地感が弱い	負荷比較用に切替可能
円形影	小	形状と方向を反映しにくい	今回の3D表現には不足
Shadow Map	深度描画を追加	モデル形状と方向を反映	採用。カリングと統計を追加

```
PlayScene.cpp
if (BeginShadowPass(pos)) {
    player->DrawShadow();
    enemies->DrawShadow();
    EndShadowPass();
}
```

2048を選んだ理由

影範囲144x144で1 texel ≒ 0.070。R32は16MiB。1024は4MiBだが輪郭が粗く、4096は64MiBかつ深度ピクセル数が4倍になるため、中間の2048を選びました。

GPU Timestamp Query - Debug x64

360 frames / frame平均 1.6936ms / Shadow平均 0.0307ms / 1.81%。Shadowは270 samples、候補1体/Passの軽い自動遷移条件です。

DirectX 12のリソース寿命管理

GPUが参照中のリソースとディスクリプタを早く再利用しないよう、フレーム単位で寿命を管理します。

FrameContext x 2



DirectXCommon.h

```

struct FrameContext {
    ComPtr<ID3D12CommandAllocator> commandAllocator;
    ComPtr<ID3D12Resource> uploadArena;
    vector<ComPtr<ID3D12Resource>> deferredReleaseResources;
    size_t uploadOffset = 0;
    uint64_t fenceValue = 0;
};
  
```

SrvManager

Freeされたインデックスはすぐ再利用せず、フェンス値と一緒に解放待ちキューへ入れます。GPU完了後に再利用リストへ戻し、使用数、残数、用途を記録します。

SrvManager::Free

```

allocated_[srvIndex] = 0;
--activeCount_;
retiredDescriptors_.push_back({ srvIndex, fenceValue });
  
```

Scene stress - 2026-06-29

PASS / 3 cycles / 360 frames / Title・Play・Result = 3・3・3 / maxSrvUsed 65。内訳: Texture2D 43、Cube 1、StructuredBuffer 20、ShadowMap 1。

周回終了時のメモリ

Working Set: 376.0 → 369.0 → 369.0MiB。Private Bytes: 510.3 → 503.2 → 503.2MiB。3週の範囲では増加が続く挙動はありませんでした。

定数バッファはSRVヒープを使わず、FrameContextのupload arenaからGPU仮想アドレスを渡します。

CSVデータ駆動・デバッグ・検証

値を変える作業とコード変更を分け、CSVの弱点は厳密な数値変換とCPUテストで補います。

CSVを選んだ理由

方法	長所	今回の判断
C++直書き	型が強い	値変更ごとにビルドが必要
JSON	階層構造に強い	表形式では記述が冗長
Binary	読み込効率を出せる	手編集と差分確認が難しい
CSV	表編集と差分確認	型チェックを追加して採用

実行中に見る値

SoftCapTelemetry / StatisticsDebugPanel

敵数、撃破数、EXP Orb、通常弾、破棄数、パーティクル、FPS履歴、ShadowPassStats、SRV使用数と用途を表示します。

strict parse

```
CsvReader::ParseFloat
const float result = std::stof(text, &parsedLength);
if (parsedLength != text.size() || !std::isfinite(result)) {
    throw std::invalid_argument("not a finite float");
}
```

1.0x、nan、桁あふれを不正値として扱います。playerStatus、weaponUpgradeSettings、enemyTypes、enemySpawnSettings、levelupWeights、ui_layout、debug_tuningをCSV化しています。

自動確認

CPU test / scene transition stress

セルキー、HP/ゲージ範囲、CSV数値変換などをCPUテストで確認。シーン遷移では360フレームのSRV内訳とメモリをCSVへ記録します。

実測: 空間分割の切り出しCPU benchmark / Shadow Pass GPU timestamp / 3周のSRV・memory推移

改善履歴と今後

実装した機能だけでなく、処理の置き場所と確認方法も制作中に見直しました。

見直した点	現在の形	変更の狙い
シーン生成	GameSceneFactoryへ集約	main.cppを起動処理に限定
ゲーム進行	GameplayFlowController	ポーズやレベルアップの条件を集約
衝突判定	OBB + 空間分割	見た目とのずれと候補増加を分けて対処
影描画	Shadow Map + culling + stats	描画経路と確認値を同時に追加
リソース寿命	FrameContext + SrvManager	GPU完了前の再利用を防ぐ
検証	CPU test + scene stress	手動プレイだけに依存しない

確認済み

- 空間分割は256体以上で全走査より短時間
- Shadow Pass平均0.0307ms（軽負荷条件）
- シーン遷移3周PASS、SRV最大使用数65
- CSV不正値を含むCPUテスト

次に計測する項目

- ゲーム内更新全体の衝突CPU時間
- 多数の敵がいる場面をPIXでも計測
- Play時間と周回数を増やしたmemory確認
- 追加アセットの起動後反映と型付きハンドル統一